PROJECT REPORT

Low-Level Design Document

RAG-Based Customer Support Assistant
with LangGraph & Human-in-the-Loop

Name: Shaik Moinuddin

Intern ID: IN126008802

Date: 22/04/2026

Document Type: Low-Level Design (LLD)

Contents

1 Module-Level Design **2**
- 1.1 Document Processing Module . . . 2
- 1.2 Chunking Module . . . 2
- 1.3 Embedding Module . . . 3
- 1.4 Vector Storage Module . . . 3
- 1.5 Retrieval Module . . . 3
- 1.6 Intent Classifier Module . . . 4
- 1.7 Graph Execution Module . . . 4
- 1.8 HITL Module . . . 5

2 Data Structures **6**
- 2.1 Document Representation . . . 6
- 2.2 Chunk Format . . . 6
- 2.3 Embedding Structure . . . 6
- 2.4 GraphState Object . . . 7

3 Workflow Design (LangGraph) **8**
- 3.1 Node Definitions . . . 8
- 3.2 Edge Definitions . . . 8
- 3.3 Graph Construction Code . . . 8

4 Conditional Routing Logic **10**
- 4.1 Routing Decision Tree . . . 10
- 4.2 Answer Generation Criteria . . . 10
- 4.3 Escalation Criteria . . . 11

5 HITL Design **12**
- 5.1 Escalation Flow . . . 12
- 5.2 Escalation Triggers . . . 12
- 5.3 Human Response Integration . . . 13

6 API / Interface Design **14**
- 6.1 Input Format . . . 14
- 6.2 Output Format . . . 14

7 Error Handling **15**
- 7.1 Error Categories and Handling . . . 15
- 7.2 Graceful Degradation Chain . . . 15

Chapter 1

Module-Level Design

1.1 Document Processing Module

File: `src/document_processor.py`
Responsibility: Load PDF files and split them into chunks suitable for embedding.

Listing 1.1: Document Processing Function Signatures

```
def load_pdf(pdf_path: str) -> list[Document]:
    """Load PDF and extract text page-by-page.
    Raises: FileNotFoundError, ValueError (empty PDF)"""

def chunk_documents(
    documents: list[Document],
    chunk_size: int = 500,
    chunk_overlap: int = 100
) -> list[Document]:
    """Split documents using RecursiveCharacterTextSplitter.
    Adds chunk_index and total_chunks metadata."""

def process_knowledge_base(pdf_path: str = None) -> list[Document]:
    """End-to-end pipeline: load -> chunk.
    If pdf_path is None, processes all PDFs in knowledge_base/."""
```

1.2 Chunking Module

Strategy: RecursiveCharacterTextSplitter

Algorithm — splitting priority order:

1. Split on `"\n\n"` (paragraph boundaries)
2. If chunks still > chunk_size, split on `"\n"` (line breaks)
3. If still too large, split on `". "` (sentence boundaries)
4. If still too large, split on `" "` (word boundaries)
5. Last resort: character-level splitting

Parameter rationale:

- **500 chars:** ≈100–125 words — enough to capture a complete FAQ answer while remaining granular for precise retrieval.
- **100 chars overlap (20%):** Prevents context loss at chunk boundaries. A sentence spanning two chunks will appear fully in at least one.

1.3 Embedding Module

File: `src/llm.py` — `get_embeddings()`

Listing 1.2: Embedding Initialisation

```
def get_embeddings() -> HuggingFaceEmbeddings:
    """Initialize local HuggingFace embedding model.
    Model: all-MiniLM-L6-v2 (runs locally, no API needed)
    Output: 384-dimensional vectors"""
```

Example embedding process:
`"How do I reset my password?"` → `[0.0234, -0.0891, 0.1456, ..., 0.0023]` (384 dimensions)

1.4 Vector Storage Module

File: `src/vector_store.py`

Listing 1.3: Vector Store API

```
def initialize_vectorstore(chunks: list, embeddings) -> Chroma:
    """Create new ChromaDB collection. Persists to ./chroma_db/
    Collection: 'customer_support_docs'"""

def get_vectorstore(embeddings) -> Chroma:
    """Load existing ChromaDB collection."""

def clear_vectorstore() -> None:
    """Delete the chroma_db/ directory."""

def get_collection_stats(embeddings) -> dict:
    """Return collection status, document count, and paths."""
```

ChromaDB Configuration:

- `persist_directory`: `./chroma_db/`
- `collection_name`: `customer_support_docs`
- `distance_function`: cosine (default for Chroma)

1.5 Retrieval Module

File: `src/retriever.py`

Listing 1.4: Retrieval API

```
def create_retriever(vectorstore, k: int = 3):
    """Create LangChain retriever with similarity search."""

def retrieve_with_scores(vectorstore, query: str, k: int = 3):
    """Return: [(Document, score), ...] sorted by relevance.
    Score range: 0.0 (irrelevant) to 1.0 (exact match)"""

def check_retrieval_confidence(results: list) -> dict:
    """Evaluate retrieval quality.
```

```
    Returns: confidence_level, scores, is_sufficient,
             needs_escalation."""

def format_retrieved_context(results: list) -> str:
    """Format docs with source attribution for LLM prompt."""
```

Confidence Evaluation Logic:

- top_score $\geq 0.5 \rightarrow$ **HIGH** confidence $\rightarrow$ proceed to generation
- top_score $\geq 0.3 \rightarrow$ **MEDIUM** confidence $\rightarrow$ proceed with caution
- top_score $< 0.3 \rightarrow$ **LOW** confidence $\rightarrow$ escalate to human
- results == empty $\rightarrow$ **LOW** confidence $\rightarrow$ escalate to human

1.6 Intent Classifier Module

File: `src/intent_classifier.py`

Listing 1.5: Intent Classifier API

```
def classify_intent(query: str, llm) -> dict:
    """Classify query into one of 6 categories using LLM.
    Returns: {intent, needs_escalation, confidence}"""

def _fuzzy_match_intent(raw_intent: str) -> str:
    """Fallback keyword matching when LLM output doesn't
    match expected categories."""

def get_escalation_reason(intent: str, query: str) -> str:
    """Generate human-readable escalation reason."""
```

Intent Categories and Routing:

Table 1.1: Intent Categories and Routing Actions

Intent	Action
GENERAL_QUERY	Proceed to retrieval $\rightarrow$ generate response
TECHNICAL_ISSUE	Proceed to retrieval $\rightarrow$ generate response
BILLING	Proceed to retrieval $\rightarrow$ generate response
COMPLAINT	Immediate escalation (skip retrieval)
ESCALATE	Immediate escalation (skip retrieval)
OUT_OF_SCOPE	Immediate escalation (skip retrieval)

1.7 Graph Execution Module

File: `src/graph_workflow.py`

Listing 1.6: Graph Node and Routing Functions

```
def classify_intent_node(state: GraphState) -> dict:
    """Node 1: Classify intent, set escalation flag."""
```

```
def retrieve_documents_node(state: GraphState) -> dict:
    """Node 2: Retrieve documents from ChromaDB."""

def evaluate_confidence_node(state: GraphState) -> dict:
    """Node 3: Score retrieval quality."""

def generate_response_node(state: GraphState) -> dict:
    """Node 4: Generate LLM response with context."""

def escalate_to_human_node(state: GraphState) -> dict:
    """Node 5: Create escalation ticket + message."""

def route_after_intent(state: GraphState) -> str:
    """Conditional edge: intent-based routing."""

def route_after_confidence(state: GraphState) -> str:
    """Conditional edge: confidence-based routing."""

def build_graph() -> StateGraph:
    """Construct and compile the full workflow graph."""

def run_graph(query: str, chat_history: list = None) -> dict:
    """Execute the graph with a query, return final state."""
```

1.8 HITL Module

File: `src/hitl.py`

Listing 1.7: HITL Module API

```
def check_escalation_needed(state: dict) -> bool:
    """Multi-factor escalation check:
    1. Intent in ESCALATION_INTENTS
    2. Retrieval confidence < threshold
    3. Explicit needs_escalation flag"""

def create_escalation_ticket(state: dict) -> dict:
    """Package context into ticket:
    {ticket_id, timestamp, query, intent, confidence,
     reason, retrieved_context, status}"""

def integrate_human_response(state: dict,
                             human_input: str) -> dict:
    """Merge human response into state,
    clear escalation flags."""

def get_escalation_message(state: dict) -> str:
    """Generate intent-specific escalation message."""
```

Chapter 2

Data Structures

2.1 Document Representation

Listing 2.1: Document Object Structure

```
# Example after loading:
Document(
    page_content="TechFlow Solutions is a leading provider...",
    metadata={
        "source": "C:/path/to/customer_support_guide.pdf",
        "page": 0,
        "source_file": "customer_support_guide.pdf",
    }
)
```

2.2 Chunk Format

Listing 2.2: Chunk Document Structure

```
Document(
    page_content="To create a new TechFlow account: 1) Visit...",
    metadata={
        "source": "C:/path/to/customer_support_guide.pdf",
        "page": 2,
        "source_file": "customer_support_guide.pdf",
        "start_index": 1245,    # Position in original page
        "chunk_index": 7,       # Global chunk number
        "total_chunks": 42,     # Total chunks in knowledge base
    }
)
```

2.3 Embedding Structure

Listing 2.3: ChromaDB Vector Entry

```
{
    "id": "doc_7",
    "embedding": [0.0234, -0.0891, ...],  # 384-dim float vector
    "document": "To create a new TechFlow...",
    "metadata": {
        "source_file": "customer_support_guide.pdf",
        "page": 2,
        "chunk_index": 7,
    }
}
```

2.4 GraphState Object

Listing 2.4: GraphState TypedDict

```
class GraphState(TypedDict):
    # Input
    query: str                          # User's raw query
    chat_history: list                  # Previous messages

    # Intent Classification
    intent: str                         # "GENERAL_QUERY" | ...
    intent_confidence: str              # "high" | "inferred"

    # Retrieval
    retrieved_docs: list                # [(Document, score), ...]
    context: str                        # Formatted context string
    sources: list                       # Source info for display

    # Confidence
    retrieval_confidence: float         # 0.0 to 1.0
    confidence_details: dict            # Full assessment

    # Response
    response: str                       # Final answer text

    # Escalation
    needs_escalation: bool              # HITL trigger flag
    escalation_reason: str              # Human-readable reason
    escalation_ticket: Optional[dict]   # Ticket data
    human_response: str                 # Agent's response
```

Chapter 3

Workflow Design (LangGraph)

3.1 Node Definitions

Table 3.1: Graph Node Definitions

Node	Responsibility	Input (State)	Output (State)	LLM?
classify_intent	Classify query intent	query	intent, needs_escalation	Yes
retrieve_documents	Search ChromaDB	query	retrieved_docs, context	No
evaluate_confidence	Score retrieval quality	retrieved_docs	retrieval_confidence	No
generate_response	Create final answer	query, context	response	Yes
escalate_to_human	Create ticket + message	query, intent	response, ticket	No

3.2 Edge Definitions

Table 3.2: Graph Edge Definitions

From	To	Type	Condition
START	classify_intent	Entry Point	Always
classify_intent	retrieve_documents	Conditional	needs_escalation == False
classify_intent	escalate_to_human	Conditional	needs_escalation == True
retrieve_documents	evaluate_confidence	Direct	Always
evaluate_confidence	generate_response	Conditional	needs_escalation == False
evaluate_confidence	escalate_to_human	Conditional	needs_escalation == True
generate_response	END	Terminal	Always
escalate_to_human	END	Terminal	Always

3.3 Graph Construction Code

Listing 3.1: LangGraph StateGraph Construction

```
workflow = StateGraph(GraphState)

# Add nodes
workflow.add_node("classify_intent", classify_intent_node)
```

```
workflow.add_node("retrieve_documents", retrieve_documents_node)
workflow.add_node("evaluate_confidence", evaluate_confidence_node)
workflow.add_node("generate_response", generate_response_node)
workflow.add_node("escalate_to_human", escalate_to_human_node)

# Set entry point
workflow.set_entry_point("classify_intent")

# Conditional edges
workflow.add_conditional_edges(
    "classify_intent", route_after_intent,
    {"escalate_to_human": "escalate_to_human",
     "retrieve_documents": "retrieve_documents"})

workflow.add_edge("retrieve_documents", "evaluate_confidence")

workflow.add_conditional_edges(
    "evaluate_confidence", route_after_confidence,
    {"escalate_to_human": "escalate_to_human",
     "generate_response": "generate_response"})

# Terminal edges
workflow.add_edge("generate_response", END)
workflow.add_edge("escalate_to_human", END)

graph = workflow.compile()
```

Chapter 4

Conditional Routing Logic

4.1 Routing Decision Tree

Figure 4.1 shows the complete conditional routing logic. The system applies two decision checkpoints — one immediately after intent classification and one after confidence evaluation.

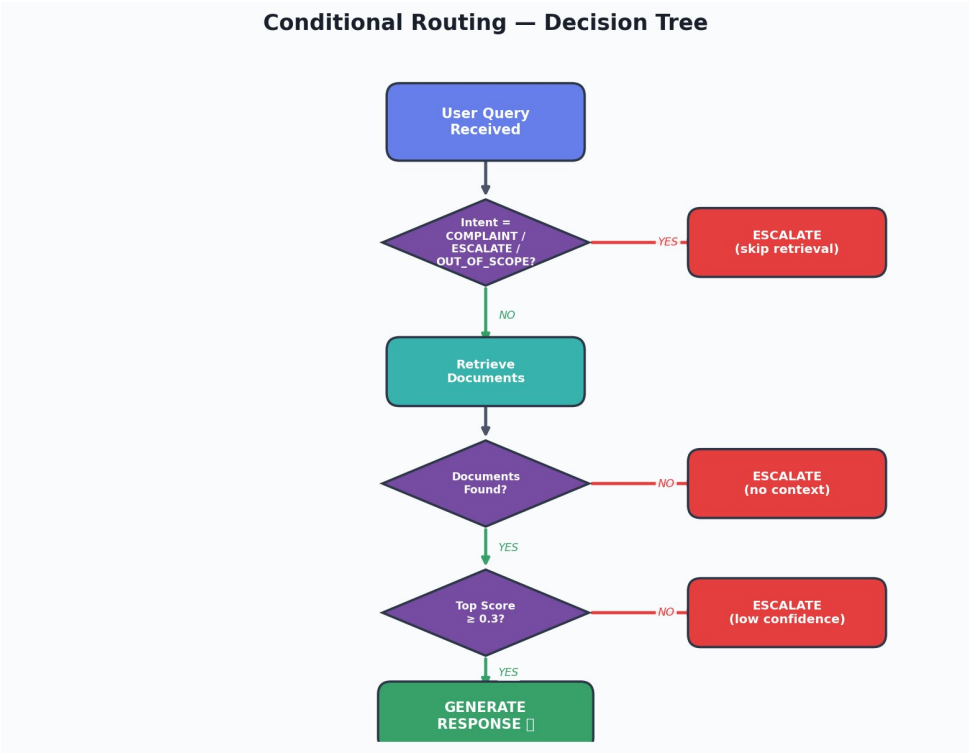

Figure 4.1: Conditional Routing — Decision Tree

4.2 Answer Generation Criteria

A response is generated automatically when **all** of the following are true:

Listing 4.1: Response Generation Predicate

```
def should_generate_response(state):
    return (
        state["intent"] not in ["COMPLAINT", "ESCALATE",
                                "OUT_OF_SCOPE"]
        and state["retrieval_confidence"] >= 0.3
        and state["needs_escalation"] is False
        and len(state["retrieved_docs"]) > 0
    )
```

4.3 Escalation Criteria

Escalation is triggered when **any** of the following conditions is true:

- **Low Confidence:** top_retrieval_score < 0.3
- **Missing Context:** no documents retrieved, or vector store not initialised
- **Complex Intent:** intent $\in$ {COMPLAINT, ESCALATE, OUT_OF_SCOPE}

Chapter 5

HITL Design

5.1 Escalation Flow

Figure 5.1 illustrates the complete HITL escalation process from trigger detection through to resuming normal flow.

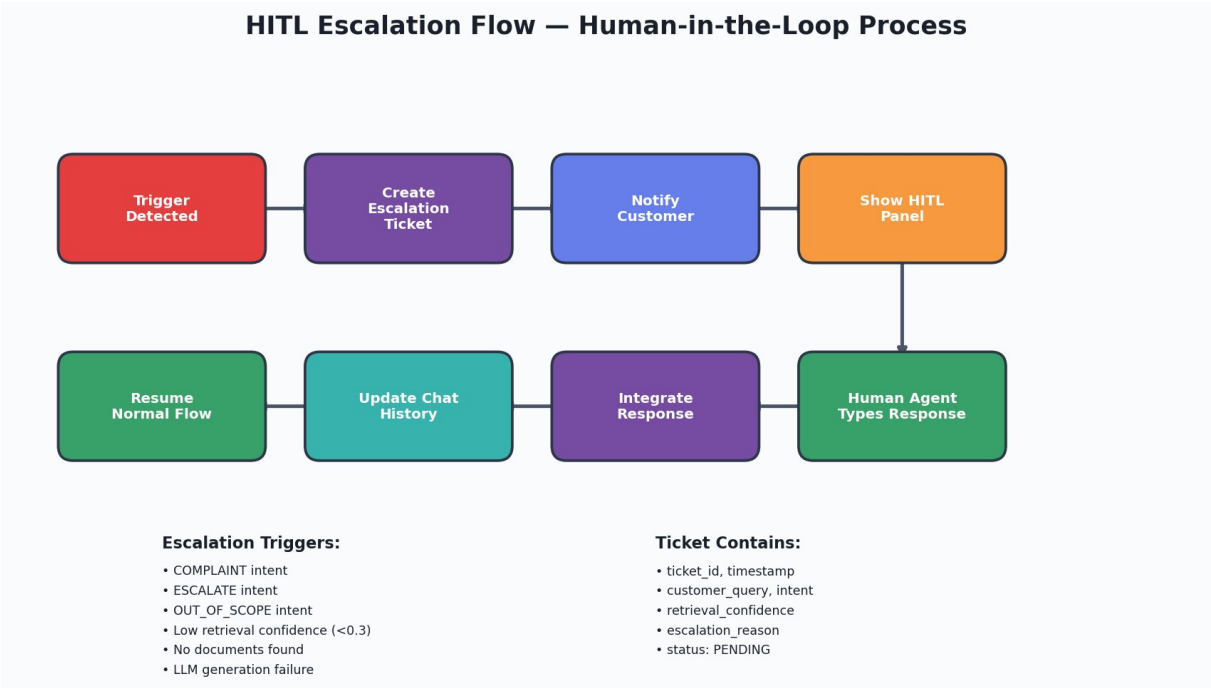

Figure 5.1: HITL Escalation Flow — Human-in-the-Loop Process

5.2 Escalation Triggers

Table 5.1: Escalation Triggers

Trigger	Condition	Priority
Explicit Request	Intent == "ESCALATE"	Immediate
Customer Complaint	Intent == "COMPLAINT"	Immediate
Out-of-Scope	Intent == "OUT_OF_SCOPE"	Immediate
Low Confidence	top_score < 0.4	After retrieval
No Documents	len(retrieved_docs) == 0	After retrieval
LLM Failure	Exception during generation	After generation attempt
KB Not Initialised	Vector store missing	Before retrieval

5.3 Human Response Integration

Listing 5.1: integrate_human_response Function

```
def integrate_human_response(state: dict,
                             human_input: str) -> dict:
    """
    1. Takes the human agent's typed response
    2. Wraps with "[Human Agent]" prefix
    3. Updates the state:
       - response = formatted human response
       - needs_escalation = False  (resolved)
       - escalation_reason = ""    (cleared)
    4. Returns updated state for chat history
    """
```

Chapter 6

API / Interface Design

6.1 Input Format

Listing 6.1: System Input Schema

```
# User provides a natural language query string
input: str = "How do I reset my password?"

# System internally builds:
{
    "query": "How do I reset my password?",
    "chat_history": [
        {"role": "user",      "content": "previous question"},
        {"role": "assistant", "content": "previous answer"},
    ]
}
```

6.2 Output Format

Listing 6.2: System Output Schema

```
# run_graph() returns:
{
    "query": str,               # Original query
    "intent": str,              # Classified intent
    "response": str,            # Answer or escalation message
    "sources": [                # Documents used
        {
            "content": str,     # Chunk text (truncated)
            "source": str,      # Filename
            "page": int,        # Page number
            "score": float,     # Relevance score
        }
    ],
    "needs_escalation": bool,    # Was it escalated?
    "escalation_reason": str,    # Why (if escalated)
    "retrieval_confidence": float,
    "escalation_ticket": dict|None,
}
```

Chapter 7

Error Handling

7.1 Error Categories and Handling

Table 7.1: Error Handling Strategy

Error	Location	Handling Strategy
PDF not found	load_pdf()	Raise FileNotFoundError with clear path message
Empty PDF	load_pdf()	Raise ValueError with extraction failure message
Vector store missing	get_vectorstore()	Raise FileNotFoundError, UI shows upload prompt
Empty collection	get_vectorstore()	Raise ValueError, suggest re-ingestion
No relevant chunks	retrieve_with_scores()	Return empty list → escalate
LLM API failure	generate_response_node()	Catch exception → escalate
Intent parse error	classify_intent()	Fallback to GENERAL_QUERY
Network error	Any API call	Caught at node level → escalate
ChromaDB corruption	get_vectorstore()	Clear and re-ingest

7.2 Graceful Degradation Chain

The system never crashes silently. Every error path leads to either: (1) a user-facing error message, (2) an escalation to a human agent, or (3) a prompt to re-initialise the system.

Listing 7.1: LLM Failure Handling in generate_response_node

```
try:
    response = llm.invoke(messages)
    return {"response": response.content}
except Exception as e:
    return {
        "response": "I apologize, but I encountered an error.",
        "needs_escalation": True,
        "escalation_reason": f"LLM generation error: {str(e)}",
    }
```